# LABORATORIO

## Evaluación de paralelismo a nivel de instrucción

## Arquitectura de Computadores

CC BY NC SA
David Expósito Singh

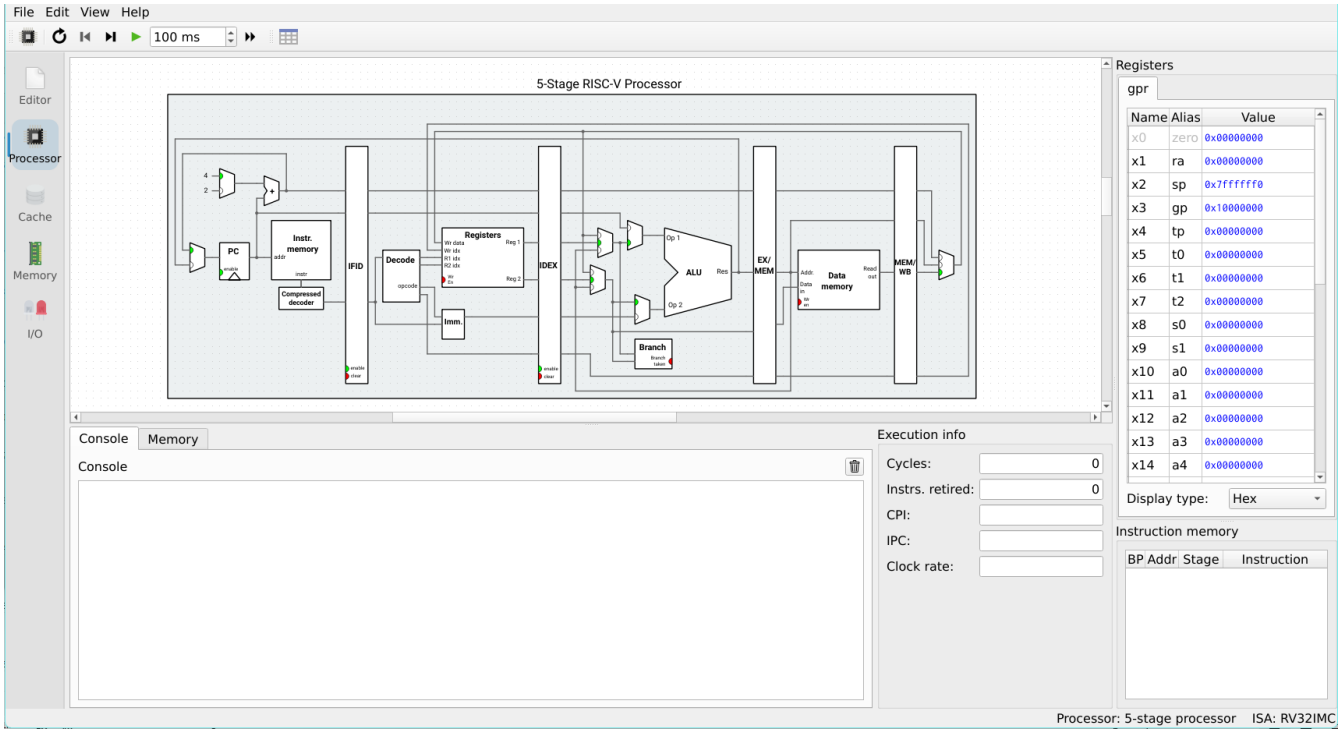
El objetivo del laboratorio es uso de Ripes, un entorno de simulación de la arquitectura RISC-V para evaluar y entender distintas configuraciones de un procesador segmentado.

## Parte 1. Descripción del entorno de simulación

Mediante el siguiente enlace se puede acceder a la versión online de Ripes.

https://ripes.dk/

La figura de abajo muestra una captura de pantalla de la herramienta.



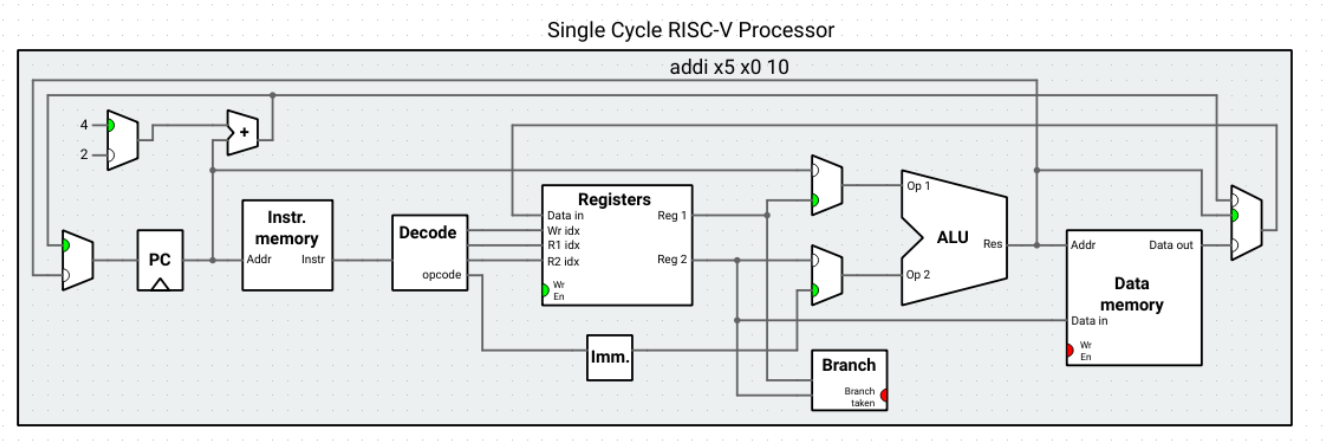

## Actividad 1: inicialización del entorno

1. Abra Ripes en un navegador web.
2. Descargue de Aula Global el archivo zip con el código de apoyo.
3. El profesor realizará una descripción del entorno del simulador.

BY NC SA

## Parte 2. Evaluación del rendimiento en una arquitectura no segmentada

Ripes incluye distintas configuraciones del camino de datos del procesador. En esta parte se evaluará la arquitectura no segmentada mostrada abajo:



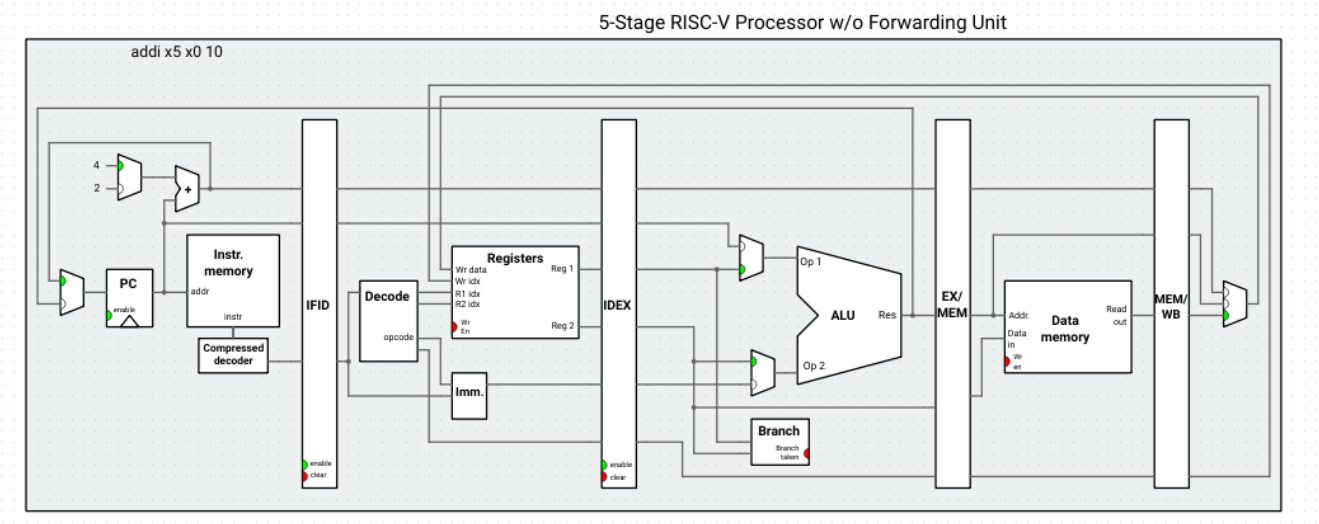

## Actividad 2: evaluación de rendimiento en una arquitectura básica

1. Configure el simulador como un procesador de un único ciclo (*single-cycle processor*).
2. Cargue el código 1 en el simulador.
3. Ejecute el código, analice su funcionamiento y resultado producido en registros.
4. Contabilice el número de ciclos, CPI e IPC.

BY NC SA

## Parte 3. Evaluación del rendimiento en una arquitectura segmentada básica

El objetivo de esta parte es evaluar el rendimiento del procesador cuando se segmenta (mostrado en la figura de abajo).



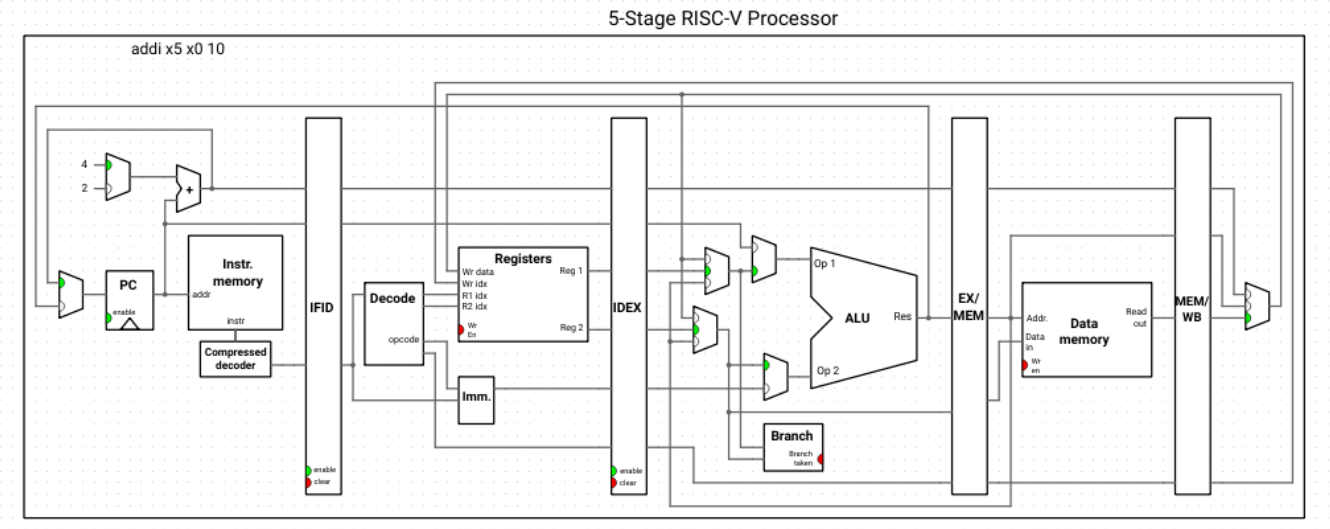

## Actividad 3: evaluación de rendimiento en una arquitectura segmentada

1. Configure el simulador como un procesador segmentado “5-stage processor w/o forwarding”.
2. Cargue el código 1 en el simulador.
3. Ejecute el código, analice su funcionamiento y resultado producido en registros.
4. Contabilice el número de ciclos, CPI e IPC.

CC BY NC SA David Expósito Singh

## Parte 4. Evaluación del rendimiento en una arquitectura segmentada con *forwarding*

El objetivo de esta parte es evaluar el rendimiento del procesador cuando se segmenta y el *forwarding* está activado (mostrado en la figura de abajo).




### Actividad 4: evaluación de rendimiento en una arquitectura con forwarding

1. Configure el simulador como un procesador segmentado “5-stage processor”.
2. Cargue el código 1 en el simulador.
3. Ejecute el código, analice su funcionamiento y resultado producido en registros.
4. Contabilice el número de ciclos, CPI e IPC.

BY NC SA

## Parte 5. Evaluación del coste predicción de saltos

El objetivo de esta parte es evaluar el rendimiento del procesador segmentado y con *forwarding* cuando tiene que realizar saltos. **Por defecto, la política aplicada es la de salto no tomado y la condición de salto y la dirección destino se calculan en la etapa de ejecución.**

### Actividad 5: evaluación de un programa con riesgos de control

1. Configure el simulador como un procesador segmentado “5-stage processor”.
2. Cargue el código 2 en el simulador.
3. Ejecute el código, analice su funcionamiento y resultado producido en registros.
4. Contabilice el número de ciclos, CPI e IPC.
5. Pensar una alternativa de código para no tener el coste de la predicción incorrecta.

BY NC SA David Expósito Singh

## Parte 6. Reordenamiento de código para evitar penalizaciones

El objetivo de esta parte es evaluar técnicas de compilación para reducir el número de detenciones en el hardware debido a dependencias de datos.

### Actividad 6: evaluación de programa con riesgos de datos RAW

1. Configure el simulador como un procesador segmentado “5-stage processor w/o forwarding”.
2. Cargue el código 3 en el simulador.
3. Ejecute el código, analice su funcionamiento y resultado producido en registros.
4. Cuente las detenciones realizas.
5. Configure ahora el simulador como un procesador segmentado “5-stage processor”.
6. Cargue el código 3 en el simulador.
7. Ejecute el código, analice su funcionamiento y resultado producido en registros.
8. Analice qué diferencias existen con el caso anterior.
9. Contabilice el número de ciclos, CPI e IPC.
10. Pensar una alternativa de código para reducir el número de detenciones.

CC BY NC SA David Expósito Singh